LaTeX2Typst Documentation

Gianfranco Baccarella e Vito Di Bari

13 April 2026

Contents

1 Introduzione ... 1
1.1 Lavori correlati ... 2
1.1.0.1 Tylax ... 2
2 Scope del traduttore ... 2
3 Struttura del progetto ... 3
3.1 Analisi Sintattica - Parsing ... 3
3.1.1 Pest ... 4
3.1.2 Errori sintattici ... 5
3.2 Analisi Semantica - AST ... 5
3.2.1 Errori semantici ... 7
3.3 Generazione del codice Typst - CodeGen ... 8
3.3.1 Funzionalità e Environment Implementati ... 9
3.3.2 Error Checking ... 9
4 Test e Risultati ... 10
4.1 Esempio Test Case ... 10
4.2 Esempio Document ... 11
5 Conclusioni ... 11

1 Introduzione

LaTeX rappresenta un potente linguaggio di markup attualmente utilizzato in numerosi contesti, soprattutto accademici. La sua potenza rappresentativa permette di realizzare di documenti di testo secondo regole di formattazione e sintassi ben definite.

LaTeX è definito come un linguaggio di markup di tipo WYSIWYM (What You See Is What You Mean), ossia all'utente è richiesto di redigere un file sorgente che sarà successivamente interpretato da un compilatore dedicato per produrre il documento finale. Questo approccio risultare può risultare troppo complesso per chi si approccia per la prima volta ad un nuovo strumento per la formattazione di testi differente dai classici programmi come Word di Microsoft.

Typst è un nuovo linguaggio di mark-up che presenta una sintassi in generale più semplice rispetto a quella di LaTeX e che vanta di una serie di funzionalità built-in, ossia disponibili senza la necessità di importare package specifici, come le immagini o i blocchi di codice.

Da questo nasce l'idea di creare un compilatore che sia in grado di tradurre un file sorgente scritto in linguaggio LaTeX nel corrispettivo scritto in linguaggio Typst. Il compilatore è stato modellato secondo un'architettura a due fasi:

- back-end: trasforma il file sorgente scritto in LaTeX in una *rappresentazione intermedia* (IR), basandosi sulle regole di sintassi proprie di LaTeX;
- front-end: trasforma la IR nel file Typst.

In questo caso specifico, la IR è stata implementata attraverso un Abstract Syntax Tree.

1.1 Lavori correlati

1.1.0.1 Tylax

Realizzato dall'utente scipenai. Si tratta di un progetto analogo, che permette di tradurre file LaTex in Typst e viceversa. Si tratta di progetto ancora in fase sviluppo, ma che si trova in una fase più avanzata rispetto a quello proposto da questo lavoro e che permette già di tradurre molte strutture sintattiche, compresa una sezione separata esclusiva per la matematica.

Tuttavia, al momento, non supporta la traduzione di alcuni comandi che sono stati gestiti dal presente lavoro, come: `maketitle` per la realizzazione del titolo e alcuni blocchi, l'environment `comment` per creare blocchi di commenti e `flush` per l'allineamento del testo.

È possibile provare una demo del software navigando verso la pagina dedicata.

2 Scope del traduttore

Prima di passare allo sviluppo vero è proprio, è stato definito un documento che delinea lo lo scope del traduttore, ossia tutti i costrutti/comandi/environent che saranno supportati in questa prima versione dello strumento LaTeX2Typst.

La lista delle feature supportate è contenuta in Tabella 1 ed è basata sull'elenco *LaTeX essentials* riportato nella documentazione ufficiale di Overleaf.

Feature	**Status**
Struttura Documento	Supportato
Metadati e Titoli	Supportato
Sezionamento	Supportato
Formattazione Testo	Supportato
Simboli Speciali	Supportato
Allineamento	Supportato
Liste ed Elenchi	Supportato
Matematica	Supportato
Codice	Supportato
Tabelle	Supportato
Figure	Supportato
Link	Supportato
Spazi e Interruzioni	Supportato

Pacchetti	Supportato
Commenti	Supportato

3 Struttura del progetto

Il software è stato scritto interamente in linguaggio Rust. L'architettura del transpiler segue una pipeline di elaborazione divisa in tre fasi principali, riportate in figura ed analizzate nei paragrafi seguenti.

Nell'ottica di rendere il software quanto più robusto possibile, si è posta particolare attenzione alla validazione dell'input per ogni fase dell'architettura. L'approccio è stato quello di continuare la traduzione in casi di errore ben definiti e circoscritti (es: commentando un comando LaTeX non supportato), altrimenti il processo viene arrestato con un errore quanto più chiaro possibile (es: il nome specificato nell'apertura di un environment LaTeX non combacia con quello in chiusura).

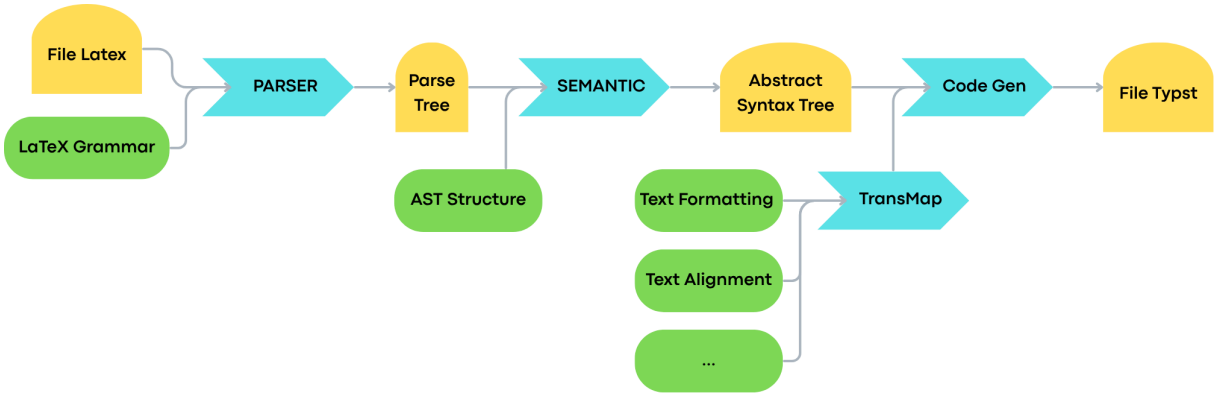

Figure 1: struttura progetto

3.1 Analisi Sintattica - Parsing

Durante questo processo, il parser si assicura che il file in input, scritto in linguaggio LaTeX (source), abbia una struttura conforme a una grammatica ben specifica. Il parser non è stato scritto a mano, bensì generato automaticamente dalla libreria Pest, un parser generator per Rust, sulla base della grammatica specificata nei file:

- `/src/latex|\_parser/latex.pest`
- `/src/latex\_parser/latex math.pest`

L'aspetto formale di Pest verrà approfondito nel paragrafo seguente.

È riportato di seguito un frammento significativo della grammatica, dove è presente lo *start symbol*`file` della grammatica prevista.

```
file = { SOI ~ item* ~ EOI }
item = { block | command | text | newlines | linebreak | comment }
text = { (!("\\" | NEWLINE | comment) ~ ANY)+ }
```

L'ordine di definizione delle regole è fondamentale in quanto il parser si fermerà al primo token non terminale che viene rispettato dal lessema analizzato, quindi per esempio: se si considera un lessema che rispetta sia la regola *command*, sia la regola *text*, il parser lo risolverà con la regola *command* poiché presente prima nel pattern.

L'output del parser sarà un ParseTree, ossia un albero che si origina da una singola radice, ordinato, e dove ogni nodo corrisponde a un match tra un lessema del file sorgente e token definito in grammatica. Ogni nodo rispetta la struttura definita dalla classe `Pair`, i cui attributi sono:

- `rule`: identificatore della regola grammaticale rispettata
- `span`:
  - `str`: coordinate esatte del lessame inteso come sottostringa all'interno dell'intero file sorgente rappresentato come un'unica superstringa
  - `range`: il range occupato dai caratteri della sottostringa
- `inner`: lista ordinata di elementi figli, quindi altre istanze di `Pair`

L'elenco delle regole grammaticali definite per il parsing del testo è riportato di seguito:

- `file`: intero testo del file di input
- `item`: componente generale del testo
- `name`: nome del comando
- `block`: blocco begin e end
- `command`: comando generale
- `linebreak`: interruzione di riga segnalata in LaTeX come doppio backslash
- `newlines`: interruzione di riga e nuove righe
- `comment`: commento monoriga
- `text`: testo semplice
- `required arg`: argomento obbligatorio del comando
- `argument`: contenuto del `required arg`
- `arg item`: tipologia di argomento di `argument`; può essere un altro comando o un testo, permettendo così il nesting dei comandi
- `arg\_text`: testo semplice di un `argument`
- `optional\_arg`: argomento opzionale del comando
- `optional\_list`: lista di argomenti che possono comporre un argomento opzionale
- `opt\_entry`: argomento singolo della lista
- `kv\_pair`: argomento chiave-valore
- `key`: chiave del `kv\_pair`
- `value`: valore del `kv\_pair`, può essere un semplice testo o una lista
- `simple\_value`: valore composto da testo semplice
- `value\_list`: valore composto da una lista di sottovalori
- `sub\_value\_list`: sottovalori della `value\_list`
- `opt\_item`: argomenti opzionali differenti dalla `kv\_pair`
- `opt\_text`: testo semplice dell' `opt\_item`
- `forbidden\_char`: insieme di caratteri da non considerare in determinate regole

3.1.1 Pest

Pest è un parser generator che utilizza le **Parsing Expression Grammars** (PEG), ossia delle grammatiche che eliminano l'ambiguità alla radice grazie all'operatore di scelta ordinata `|`: il parser tenta di far corrispondere le espressioni nell'ordine in cui sono definite, selezionando la prima che ha successo.

Le PEG, da definizione, adottano il principio di *packrat parsing* tipicamente presente nell'approccio PEG Tuttavia Pest, non adotta

Pest non implementa il backtracking quindi durante l'analisi, un'espressione o ha successo o fallisce: se ha successo, il passo successivo viene eseguito normalmente, ma se fallisce, l'intera espressione fallisce e il parsing verrà interrotto risultando in un errore bloccante per il software.

Citando la documentazione di Pest: "in un sistema con backtracking, si tornerebbe indietro di un passo, "non mangiando" un carattere, e poi si riproverebbe. Ma i PEG non fanno questo. Nella regola *first - second*, una volta che *first* viene analizzato con successo, ha consumato alcuni caratteri che non torneranno mai indietro. *second* può essere eseguito solo sull'input che *first* non ha consumato." Il parsing attutato tramite pest può essere riassunto cosi: prova a far rispettare la prima espressione, se ha successo continua continua a far rispettare il pattern finché non si conclude, se non ha successo prova con un'altra espressione. Se nessuna espressione ha successo allora genera errore.

Tornando all'esempio della grammatica prima mostrato: come possiamo vedere un file può essere composto da zero o più di un item, la definizione delle regole segue delle espressioni molto simili alle regex, ed a sua volta l'item deve essere composto da uno solo dei diversi token definiti. Item definisce quindi una regola/pattern generale che identifica ogni elemento presente nel file, successivamente si entra nello specifico di questi token come nella regola *text* che definisce come testo qualsiasi elemento che contenga uno o più caratteri che non inizi con il *backslash*, altrimenti verrebbe identificato con il token del comando, e che non rispetti la regola *NEWLINE* o *comment*.

3.1.2 Errori sintattici

Affinché il processo di traduzione sia robusto, il sorgente LaTeX dev'essere ben formato e rispettare la grammatica. Se il parser incontra un lessema che non riesce a risolvere, l'intero processo viene bloccato e viene prodotto un messaggio di errore in console che fa riferimento alla porzione di input che lo ha generato.

Nella fase di parsing viene effettuato il controllo dei soli *errori sintattici* (o *errori grammaticali*), ossia la sola correttezza sintattica della sequenza di lessemi rispetto alle regole formali definite nella grammatica PEG. Errori come comandi non riconosciuti oppure argomenti errati sono detti *errori semantici* e saranno identificati più avanti nel processo di traduzione, dove si hanno più informazioni di contesto sull'intero file sorgente.

Alcuni esempi di errori sintattici:

- parentesi graffe di un argomento incomplete, solo aperta o solo chiusa
- solo apertura o sola chiusura di un environment
- apertura e chiusura di due environment differenti
- caratteri numerici nella definizione di un environment

3.2 Analisi Semantica - AST

Si tratta di uno stadio più avanzato dell'analisi, dove è possibile interpretare le istruzioni scritte in LaTeX e verificarne la loro correttezza in base al contesto in cui si trovano, andando oltre la mera grammatica.

Per rendere più agevole la fase di analisi semantica, il ParseTree ottenuto precedentemente viene sintetizzato in un *Abstract Syntax Tree* (AST). L'AST è dunque il formato individuato per costruire la *rappresentazione intermedia* del file sorgente. A differenza del ParseTree, l'AST è molto più sintetico, espressivo e leggibile, seppur mantenendo lo stesso contenuto informativo del file originario.

Dire che l'AST sia più espressivo del ParseTree sta a significare che i nodi inglobano più informazione al loro interno che può essere di carattere locale (es: informazioni derivati dal fatto che

un certo nodo è individuato all'interno in un ambiente LaTeX) o globale (es: informazioni relative al tipo di documento definito a monte del file sorgente).

Viene quindi effettuato un processo di decorazione in cui vengono aggiungente o semplificate informazioni come ad esempio il numero di nuove righe che nel ParseTree vengono rappresentate semplicemente come testo "n", differentemente dall'AST in cui l'oggetto **NewLineNode** possiede un attributo **count**. Ad esempio la stringa **titleLaTeX To Typst** che nel ParseTree viene scomposta in 7 nodi ognuno padre dell'altro, differentemente nell'AST é rappresentato con solo 3 nodi.

- **CommandNode**, che possiede gli attributi
  - **Name**, il nome del comando
  - **OptionalArgs**, gli argomenti opzionali di un comando racchiusi tra parentesi quadre
  - **RequiredArgs**, gli argomenti obbligatori di un comando racchiusi tra parentesi graffe, in questo caso contiene un solo **RequiredArgNode**, che a sua volta possiede un attributo items di tipo **Text**
    - **TextNode**, contiene il valore testuale all'interno delle parentesi quadre.

Se la precedente struttura ad albero era basata sulla grammatica, questa nuova struttura ad albero che andiamo a realizzare è basata su una lista di strutture ed enumerativi da noi definiti, nel file *ast structure*, che vengono assegnati ai vari nodi in base alle loro regole grammaticali d'origine.

A titolo di esempio, viene riportata la struttura di un nodo di tipo `CommandNode` dell'AST:

```
pub struct CommandNode {
    pub name: String,
    pub optional_args: Vec<OptionalArgNode>,
    pub required_args: Vec<RequiredArgNode>,
}
```

Gli attributi `optional args` e `required args` sono i vettori che contengono le rispettive liste di argomenti opzionali e obbligatori accettati dal comando, modellati da altre classi.

È evidente quanto sia più compatto e pratico da gestire rispetto al nodo di tipo `Pair` visto precedentemente nel *ParseTree*.

Nella definizione della struttura dell'AST sono stati definiti 15 tipi di nodi (strutture ed enumerativi):

- `AstDocument`: nodo radice - struct
- `AstItemNode`: nodo generale per una regola - enum
- `BlockNode`: blocco begin/end - struct
- `TextNode`: testo semplice - struct
- `WhitespaceNode`: spazi bianchi che precedono testo - struct
- `NewLinesNode`: numero di nuove righe - struct
- `LinebreakNode`: numero di interuzzione di riga - struct
- `CommentNode`: commenti mono riga - struct
- `CommandNode`: comandi latex - struct
- `RequiredArgNode`: argomenti richiesti dai comandi nelle parentesi graffe - struct
- `ArgItemNode`: oggetto presente nell'argomento richiesto - enum
- `OptionalArgNode`: argomenti opzionali dei comandi nelle parentesi quadre - struct
- `OptionalEntryNode`: oggetto presente nell'argomento opzionale - enum
- `KvPairNode`: argomento definito da una chiave ed un valore - struct
- `OptValueNode`: oggetto presente come valore nel KvPairNode - enum

- `OptItemNode`: oggetto presente nell'argomento opzionale - enum
- `MathNode`: nodo contente la matematica - struct

Inoltre, sono stati definiti gli errori semantici come enumerativi. Essi vengono stampati in console in caso di errore durante la fare di generazione dell'AST: Ecco la trasformazione dei tuoi nodi di errore in una lista LaTeX pronta per essere inserita nel tuo documento:

- `MissingFileNode`
- `MissingItemChild`
- `MissingCommandName`
- `MissingBlockName`
- `MissingRequiredArgItems`
- `MissingOptionalArgEntries`
- `MissingOptionalEntryItems`
- `MissingKeyInKvPair`
- `MissingValueInKvPair`
- `EmptyTextValue`
- `EmptyCommentValue`
- `InvalidNewlineCount`
- `InvalidLinebreakValue`
- `UnexpectedItemRule(Rule)`
- `UnexpectedArgItemRule(Rule)`
- `UnexpectedOptItemRule(Rule)`
- `UnexpectedOptionalEntryRule(Rule)`
- `UnexpectedRule(Rule)`

3.2.1 Errori semantici

Durante la costruzione dell'AST può succedere che il software possa provare ad estendere l'albero aggiungendo un nuovo nodo foglia che però non rispetta la logica della grammatica; ad esempio aggiungiamo un nodo **text**, che serve a rappresentare un testo semplice, come nodo figlio di un **command**. Questa sarebbe un incoerenza nella costruzione dell'AST che potrebbe ad errori nella fase di generazione del codice typst, perciò durante la costruzione effettuiamo sempre controlli per verificare che il nodo che stiamo andando a costruire rispecchi una delle possibili regole selezionate per quel determinato contesto, altrimenti rilascia un errore di **UnexpecxtedRule**. Oltre ad un controllo della corretta regola, in questa fase verifichiamo il rispetto di altre condizioni che altrimenti genererebbero problemi nelle fasi successive, come l'assenza del nome del comando oppure, un numero invalido di nuove righe o l'assenza degli argomenti richiesti da un comando. Per quest'ultima tipologia di errore lanciamo un warn sul terminale insieme ad un errore di **MissingRequiredArgItems**, questa situazione però viene gestita diversamente in base al comando poiché in LaTeX possiamo avere comandi senza argomenti richiesti o con più di uno. Perciò effettuiamo 2 controlli, il primo durante la costruzione del comando per verificare che un comando che necessita di un argomento, come ad esempio textbf, abbia almeno un nodo figlio nel ParseTree che rispetti la regola del **required args**, in caso negativo il warn ci avvisa che si non si é trovati alcun argomenti. Differente durante la costruzione del required args si controlla che che le parentesi graffe siano popolate, quindi che sia presente una stringa, e non siano vuote. Per effettuare questi controlli é stata creata un'hashmap simile alla TransMap, che restituisce il numero di argomenti minimi attesi per tale argomento, in modo da comprendere se l'assenza di parentesi graffe risulta essere un errore oppure é un comando corretto senza argomenti.

```
if let Some(expected) = reqarg_count(&name) {
    if required_args.len() < expected as usize {
        warn!("Comando \\{}: expected {} required arguments,
            found {}", name, expected, required_args.len());

        return Err(SemanticError::MissingRequiredArgItems);
    }
}
```

3.3 Generazione del codice Typst - CodeGen

Come introdotto, l'ultima fase della traduzione é la generazione del nuovo linguaggio mantenendo la semantica invariata; vogliamo quindi lo stesso risultato da parte del nuovo linguaggio. Per fare ciò prendiamo in input l'Abstract Syntax Tree e tramite la navigazione di esso, foglia per foglia, generiamo un codice equivalente, effettuiamo un "rendering"; che nel nostro caso assume il significato di scrivere la traduzione nel file di output. Il rendering del nuovo codice avviene iterando una funzione di mapping su ogni item dell'AstDocument raccogliendo i risultati che saranno mano mano aggiunti al file. Per ogni nodo foglia su cui iteriamo effettuiamo una funzione di mapping differente poiché bisogna gestire le differenti casistiche; questa scelta é effettuata in base al rispettivo enumerativo del nodo foglia. Un esempio semplice é il rendering del nodo TextNode il cui risultato della funzione di mappamento é una semplice stringa da cui vengono rimosse le parentesi graffe poiché in latex queste non vengono renderizzate nel file pdf differentemente da typst in cui invece compaiono nel documento finale.

```
pub(crate) fn render_text(text_node: &TextNode) -> String {
    text_node
        .value
        .chars()
        .filter(|c| *c != '{' && *c != '}')
        .collect()
}
```

Differentemente per situazioni più complesse come i blocchi ed i comandi é stata implementata un hashmap, denominata **TransMap**, in cui la chiave, una stringa, é rappresentata dal nome del commando e il valore é la specifica funzione per trattarlo, questa funzione prende come argomenti:

- **name**, stringa contente il nome del comando
- **reqs**, vettore di RequiredArgNode
- **opts**, vettore di OptionalArgNode
- **items**, vettore di AstItemNode. Presente solo nelle funzioni della TransMap dedicata ai blocchi, in modo da passare anche le componenti all'interno dei blocchi e poterle trattare con le corrette funzioni poiché ci troviamo all'interno di un environment differente.

Modellando in questa maniera é possibile avere due sole mappe di riferimento per poter indirizzare verso la funzione di rendering corretta in base alla tipologia di funzione ed environment latex. Le due TransMap, avendo la stessa funzione, sono costruite seguendo lo stesso **"trait"**, che corrisponde ad una comune "interfaccia", implementando la funzione **translate** che restituisce il risultato del rendering della singola componente da aggiungere in coda alla traduzione.

```
pub trait TransMap<T> {
    fn translate(node: &T) -> Option<String>;
}
```

3.3.1 Funzionalità e Environment Implementati

Chiaramente a causa del gran numero di funzioni presenti in LaTeX, per il progetto ci siamo posti l'obiettivo di implementare solamente quelle fondamentali, di uso comune, in modo da avere un software funzionante sui documenti più semplici. Le funzionalità implementate sono:

- **Text Formatting**: bold, italic, underline, color
- **Text Alignment**: centering, raggedright/left, flushright/left
- **Space and Breaks**: newline, break, hfill/vfill, pagebreak, newpage, clearpage
- **Package**: ragged2e, verbatim, hyperref, listings, graphicx
  - questi pacchetti non essendo necessari in typst, poiché coperti da funzioni built-in, sono semplicemente renderizzati come commenti.
- **Section**: section, subsection, subsubsection, paragraph, subparagraph, title, author, date, today, maketitle, tableofcontents
- **Hyperlinks**: href

Invece gli environment implementati sono:

- **Alignment**: center, flushright/left
- **Table**: tabular
- **Comment**: commenti multilinea
- **Text Listing**: liste itemize e enumerate, description
- **Graphics**: figure
- **Code**: verbatim, lstlisting

3.3.2 Error Checking

Può succedere che il file di input presenti dei comandi la cui gestione non é ancora stata implementata oppure assenti in typst, quindi un tentativo di gestione porterebbe a generare un errore durante il rendering e quindi nel conseguente arresto del software che non restituirebbe nulla in output, questo perché al momento la scrittura sul file di output avviene tutta in unica volta a fine rendering e non riga per riga. Per evitare tutto ciò quindi sono stati implementati due controlli: uno per i comandi non ancora implementati nella traduzione o erroneamente scritti ed uno per quelli non presenti in typst. Il primo controllo viene effettuato durante il rendering generico di un comando, quindi quando si cerca la corretta funzione da chiamare, attraverso la TransMap, nel caso non sia presente alcuna key valida, quindi: o il comando desiderato non é ancora stato implementato oppure é stato scritto erroneamente, viene lanciato un warn sul terminale che avvisa l'utente dell'errore ed inoltre renderizza il comando riscrivendo lo stesso messaggio di errore anche sul file di output in modo tale da lasciare una traccia nel file di output dell'errore e poterlo correggere.

```
pub(crate) fn render_command(command_node: &CommandNode) -> String {
    if let Some(rendered) = CommandTransMap::translate(command_node) {
        rendered
    } else {
        let out = String::new();
        drop_command_warn(
            COMMANDWARNING::WrongCommandOrNotImplemented(
                command_node.name.clone()),
            Option::from(out),
            Option::from(&*command_node.name),
            Option::from(command_node.required_args.clone()),
        )
```

```
    }
}
```

Differentemente quando il comando non ha una diretta trasposizione su typst, la TransMap, che presenterà tale comando tra le sue key, restituirà la funzione in cui sarebbe dovuto essere implementato gestendo il comando come avviene nella prima situazione: warn sul terminale e comando commentato sul file di output.

4 Test e Risultati

Per verificare il corretto funzionamento del software sono stati definiti dei test d'integrazione per ogni componente della CodeGen e per le situazioni che generano errore, in modo da poter controllarne la corretta gestione. Possiamo quindi distinguere due casi di esecuzione del software:

- **Doc Case**, il caso base in cui il software effettua la sua normale esecuzione di transpiler traducendo il documento latex in input in un file typst e il rendering in pdf.
- **Test Case**, si esegue un rendering di differenti file di input suddividisi per categoria del comando in modo da avere singole unità con cui é più facile verificare il corretto funzionamento della relativa componente che si occupa della generazione del codice typst.

4.1 Esempio Test Case

Il testing si basa su un controllo del risultato dell'esecuzione di ogni singola componente che prende in input un file che presenta lessemi che ci aspettiamo debbano essere gestiti solamente da quella funzione. Questo controllo si basa sul confronto del risultato con il risultato atteso rappresentato da un file typst che dovrà essere coincidere con l'output per poter asserire il corretto funzionamento. Questa logica vale sia per le componenti di traduzione, sia per le funzioni di gestione degli errori. Per fare un esempio di input-output delle componenti di traduzione, prendiamo come caso i comandi per la formattazioni del testo. Input Latex:

```
\textbf{bold}
\textit{italic}
\underline{underline}
\textcolor{blue}{This is \textbf{bold and blue}.}
```

Output Typst:

```
*Bold*
_italic_
#underline[underline]
#text(blue)[This is *bold and blue*.]
```

Nel caso invece della gestione degli errori, nei file di output é atteso un commento che presenta il messaggio d'errore contenente la tipologia d'errore e il lessema che ha generato l'errore. Input Latex:

```
\begin{document}
    \commandoInesistente{argomento inesistente}
\end{document}
```

Output Typst:

```
/*WARN: WrongCommandOrNotImplemented("commandoInesistente"):
commandoInesistente \commandoInesistente{argomento inesistente}*/
```

Invece un esempio di input che genera un errore che blocca l'esecuzione é: Input Latex:

```
\begin{document}
    apertura e chiusura di environment differenti
\end{article}
```

Output sul terminale:

```
------------------------- PARSE ERROR -------------------------
Errore di parsing LaTeX alla riga 9, colonna 1
\end{article}
^
Atteso: item
-------------------------
```

4.2 Esempio Document

Un esempio di documento latex tradotto in typst é questo stesso documento che é stato originalmente scritto in latex e poi tradotto e renderizzato in pdf dal software realizzato.

5 Conclusioni

Il presente elaborato ha delineato la struttura e il funzionamento del software e si può concludere che il software é funzionate e svolge correttamente il compito per cui é stato designato, chiaramente allo stato attuale il software é ancora incompleto poiché non tratta molti comandi presenti in latex ed argomenti opzionali per personalizzare i comandi già implementati.

Sviluppi futuri potranno essere sicuramente l'aggiunta dei nuovi comandi nei possibili limiti del rendering di typst.